

VS Code Copilot 實踐指南：從人機互動到 Agent 底層運動邏輯

/apply-fix

```
iastert vs.cool'
import "sppe.zs";
const {foo} == {

<div class="agentssvcode-revieu">
  <agentz0ubduen9tapflzeoor:'eebnco? />
  <cocsss10G0B0Revieefeypecx{lltoot-/code-review}>
</div>

<div class="/code-reviewt:opentrucade-revice">
  <epit:rexienfesatopfi{code"Reeinpo}} />
  /apply-restieu />
</div>

}
```


核心架構：大腦與手腳的雙引擎分離

Prompt 層 (大腦 / 規則定義)

code-review.prompt.md

定義 PEP8、Docstring、SQL 規範

apply-fix.prompt.md

定義修正規則、安全檢查、保守策略

特徵：數據團隊可隨時使用自然語言修改擴充



Agent 層 (手腳 / 執行邏輯)

review.agent.md

啟動門檻檢查 → 逐檔審查 → 產出結構化報告

apply-fix.agent.md

解析 diff 區塊 → 安全驗證 → apply_patch 套用

特徵：封裝底層 YAML 與 VS Code 互動邏輯，無須更動

核心洞察：規則與邏輯完全解耦，數據團隊只需維護 Prompt (.md) 即可更新審查標準，無需動到核心邏輯。

基礎建設：核心檔案與實體空間結構

```
.github/  
├── prompts/  
│   ├── code-review.prompt.md  
│   └── apply-fix.prompt.md  
└── agents/  
    ├── review.agent.md  
    └── apply-fix.agent.md
```

Code_review_result.md

Prompt 檔

存放於 prompts 資料夾。存放所有業務邏輯、代碼風格與安全規範。

Agent 檔

存放於 agents 資料夾。掌控執行流程與底層工具呼叫。

動態 Output

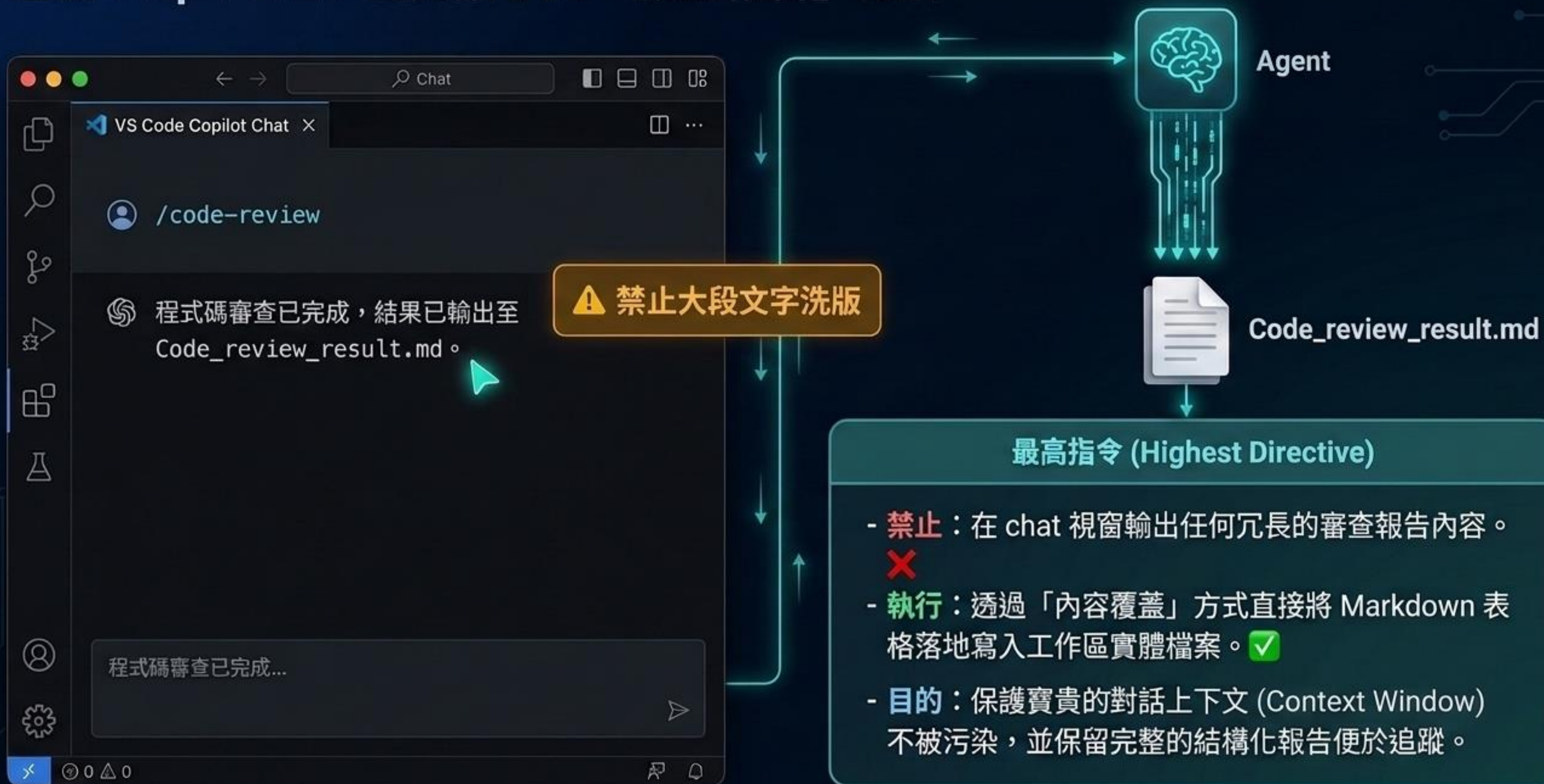
由 Agent 在根目錄動態生成的實體報告檔案，即時覆寫更新。

i 未來實踐：開發者只需直接從 Repo Pull 取得此 .github 資料夾，即可享有完整基礎建設。

全景操作流程：人機協作閉環



透視 Step 1 & 2：最高指令與「靜默報告」機制



底層魔法：YAML Formatter 與能力賦予

```
name: Apply-Fix
tools: ['read', 'edit', 'search', 'agent', 'todo']
handoffs:
  - label: 重新執行code review
    agent: Review
    prompt: /code-review
```

工具呼叫能力

賦予 Agent 讀寫實體檔案的超能力，其中 'edit' 與 'apply_patch' 是讓它能夠直接修改程式碼的關鍵。

Handoff機制

定義 Agent 之間的任務交接。當 Apply-Fix 完成後，立刻交棒給 Review Agent 重新掃描。

洞察：YAML Frontmatter 是 Copilot 認識 Agent 身份、權限與操作範圍的專屬身分證。

判定邏輯：規則映射與保守掃描機制



透視 Step 3：結構化報告解剖 (Anatomy of the Report)

Code_review_result.md

```
## Overall

Status: Fail ❌
Coverage: 100%

## Coverage

- Status: Fail ❌
- Parsensed: 100%
- Pactor scones: 75%
- Total metric: 28
```

Rules 狀態表格

規則類別	規則等級	狀態	詳細說明與修正建議
Authex	MUST	MUST Pass ✅	詳細說明專案狀態出規則滿足情況。
規則類別	MUST	MUST Fail ❌	詳細說明專案狀態出規則滿足情況。
規則類別	MUST	MUST Fail ❌	詳細說明專案狀態出規則滿足情況。

```
... @@ -223,23 @@ with bx app_retry.py
21 28         setstate = true
22 23
23 24         not all(key in app_retry.py
24 - if not all(key in app_retry.py.resysf1(et), 3)
25 + if not all(key in app_retry.py.retryef1(st), 3)
25 26
26 27         else:
27 28             selfrte_key = app_retry.py.moin()
```

區塊 1: 狀態與覆蓋範圍

顯示整體專案狀態與掃描覆蓋率，確立 100% 無遺漏。

區塊 2: Rules 狀態表格

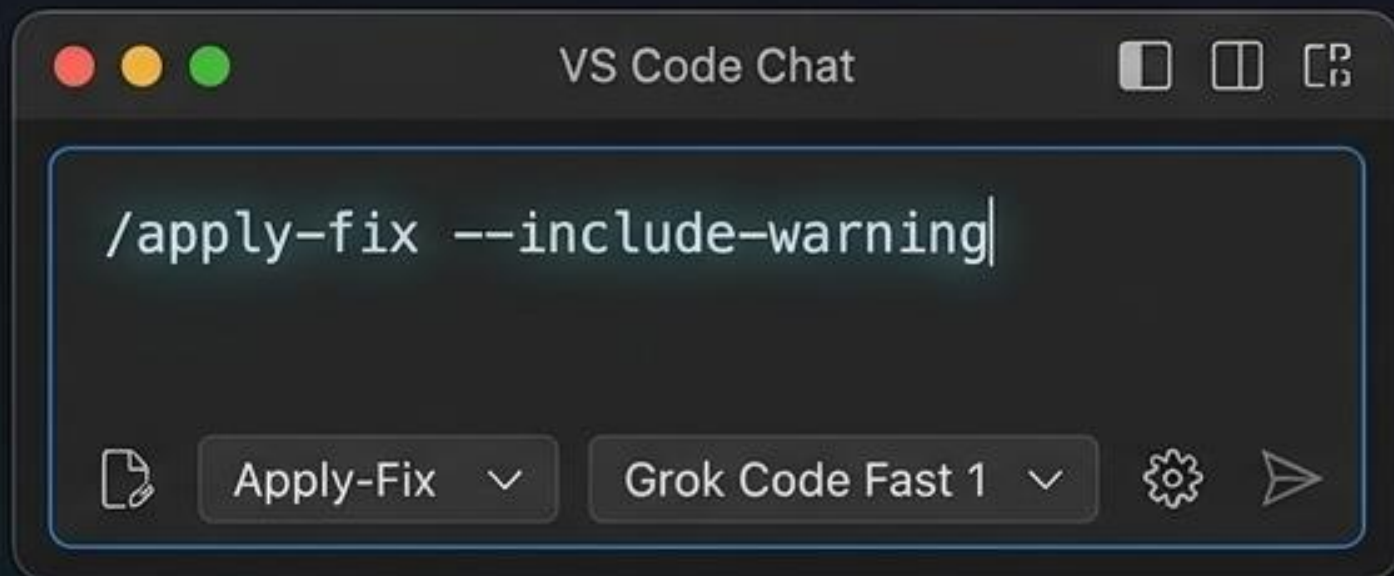
以表格一目了然列出規則滿足情況，標示 Fail❌ 與 Pass✅。

區塊 3: Unified Diff 修正區塊

精確高亮 - (刪除) 與 + (新增) 的程式碼，為下一步自動套用做標準化準備。

透視 Step 4：觸發自動修正與參數控制

精準的終端使用者操作



底層防護：Diff 解析與實體套用防護網

```
@@ -8,7 +8,10 @@  
import domain_jobs
```

Unified Diff 原始資料 (來自 .md 報告)

Agent 安全檢查
(Safety Nets)



- 拒絕絕對路徑 (如 C:\) ✖
- 拒絕路徑穿越 (禁止使用 ..) ✖
- 拒絕建立/刪除檔案 (忽略 /dev/null) ✖

安全寫入工作區 (apply_patch 工具)

嚴格遵守 diff 內容，只做指定修改。不做猜測性重構，不做『順便優化』，確保原始碼絕對安全。

代理人接力：Handoff 驗證閉環



核心洞察：透過 Agent 互相呼叫，實現「審查 → 修正 → 驗證」的無縫自動化閉環，大幅減少手動輸入指令的摩擦力。